



INDIA.RUNS

Build what next India runs on

Team Name : AutoVault Intelligence

Team Leader Name : Vatsal Agarwal

Problem Statement: Traditional talent acquisition tools rely flatly on global semantic embedding distance matching, which is easily exploited by unqualified candidates who "stuff" keywords from Job Descriptions (JDs) directly into raw resume text, creating extensive manual tracking overhead and massive talent-funnel noise.

Solution Overview

What is your proposed solution?

- **TrueBoard Ranking Engine:** A client-side, 7-layer behavior-aware candidate ranking system designed to screen and sort over 100,000 profiles against a Job Description in under 30 seconds.
- **7-Layer Processing Pipeline:** Cascades sequentially through a Semantic Base, Title Semantic Gates, Skill Authenticity Checks, Experience Layering, Behavioral Multipliers, Proctored Test Integration, and Monotonic Normalization.
- **High-Speed Local Execution:** Runs vectorized matrix multiplications entirely on the local device using pre-calculated binary .npy arrays, ensuring zero cloud-inference costs and total data privacy.

What differentiates your approach from traditional candidate matching systems?

- **Traditional Screeners (AI-Assisted):** Rely flatly on global keyword density or basic semantic similarity. They are easily gamed by unqualified applicants who copy-paste the job description text directly into their resumes.
- **TrueBoard Engine (AI-Native):** Treats resumes as chronological timelines rather than static text dumps to block profile manipulation:
 - **Skill Authenticity Verification:** Scans the text of past work experience to confirm skills were actually used on the job, completely ignoring isolated, stuffed skill lists.
 - **Title Semantic Gate:** Applies heavy mathematical penalties if a candidate's actual job title history does not match the required seniority tier.
 - **Behavioral Trust Integration:** Blends self-reported resume text with live platform data, boosting candidates with high response rates and verified proctored evaluation scores.

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

- **Technical Skills:** Core domain frameworks, programming languages, and toolsets.
- **Seniority Tier:** Required target job titles and role organizational levels.
- **Experience Thresholds:** Mandatory baseline and preferred Years of Experience (YoE) boundaries.
- **Testing Benchmarks:** Required proctored assessment types and milestone verification targets.

Which candidate signals are most important for determining relevance?

- **Chronological History:** Narrative text descriptions from past employment blocks.
- **Title Trajectory:** Historical alignment of job titles against target seniority.
- **Platform Engagement:** Real-time user response rates and activity recency indicators.
- **Verified Metrics:** Authenticated proctored test scores over self-reported resume text.

How does your solution evaluate candidate fit beyond keyword matching?

- **Title Semantic Gate:** Applies heavy math penalties if a candidate's job title history matches a junior or adjacent role rather than the target tier.
- **Skill Authenticity Check:** Verifies skills exist within past work experience text, ignoring isolated, keyword-stuffed skill lists.
- **Behavioral Trust Loop:** Combines text matching with platform activity logs, boosting responsive candidates with verified assessment milestones.

Ranking Methodology

How does your system retrieve, score, and rank candidates?

- **Vector Matrix Extraction:** Maps the input JD text and runs ultra-fast matrix operations against 100,000+ pre-cached profile vectors.
- **Layered Score Adjustment:** Feeds top candidates through cascading rules to apply math penalties for title mismatches and bonuses for verified experience.
- **Monotonic Sorting:** Flattens multi-dimensional candidate data arrays into a single list sorted from 1.0 down to 0.0.

What models, algorithms, or heuristics are used?

- **Semantic Embeddings:** Uses a local SentenceTransformer (all-MiniLM-L6-v2) for fast, compact text vectorization.
- **Title/Experience Heuristics:** Employs conditional penalization and boosting rules based on string alignment metrics.
- **Engagement Multipliers:** Scales scores using live platform flags like response rates and active profile timelines.
- **Normalization Algorithm:** Applies monotonic mapping functions to guarantee stable, predictable sorting outputs.

How are multiple candidate signals combined into a final ranking?

- **Unified Pipeline Formula:** Computes the baseline text match, modifies it via structural career checks, and scales the subtotal by engagement signals:

$$S_{\text{final}} = ((S_{\text{base}} \times M_{\text{title gate}}) + B_{\text{experience}}) \times M_{\text{behavior}} \times M_{\text{proctor}}$$

- **Stable Index Normalization:** Compresses the final output scores into a standardized 0.0 to 1.0 distribution range to fit strict enterprise integration needs.

Explainability & Data Validation

How are ranking decisions explained?

- **Layer-by-Layer Auditing:** Breaks down the final candidate score into an explicit, transparent 7-layer scorecard (e.g., displaying exact base similarity, title gate penalties, or behavioral bonuses applied).
- **Traceable Component Attribution:** Recruiters can verify exactly which criteria influenced a candidate's rank, making the platform's decision logic clear and verifiable rather than an opaque "black box".

How do you prevent hallucinations or unsupported justifications?

- **Zero Generative Logic:** Completely isolates candidate ranking from generative AI text layers, bypassing LLM summary generation entirely.
- **Deterministic Mathematical Scoring:** Relies exclusively on mathematical vector calculations and factual database metadata, ensuring 100% stable, data-backed sorting decisions.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

- **Skill Authenticity Penalization:** Drastically drops scores for keyword-stuffed resumes by verifying skill context inside chronological employment history blocks, ignoring isolated lists.
- **Title Gate Disconnect Isolation:** Instantly catches role mismatches via the Title Semantic Gate, filtering out deceptive or unrelated backgrounds regardless of profile keyword density.
- **Behavioral Verification Layering:** Prioritizes active platform interaction logs and authenticated, proctored test milestones, automatically pushing stagnant or suspicious profiles to the bottom.

End-to-End Workflow

1. Job Description Ingestion & Parameter Mapping

- The enterprise recruiter uploads the target Job Description (JD) text block directly into the Flask application interface.
- TrueBoard instantly parses the requirements, mapping core technical frameworks, required job titles, baseline experience bounds, and target proctored assessment milestones into an in-memory extraction object.

2. Matrix Cosine Similarity Lookups

- The engine calls the local all-MiniLM-L6-v2 SentenceTransformer model to convert the parsed JD requirements into a dense text vector.
- It executes high-speed matrix multiplications across 100,000+ candidate profile vectors loaded from local binary .npy storage caches, calculating base semantic similarity scores in under 30 seconds.

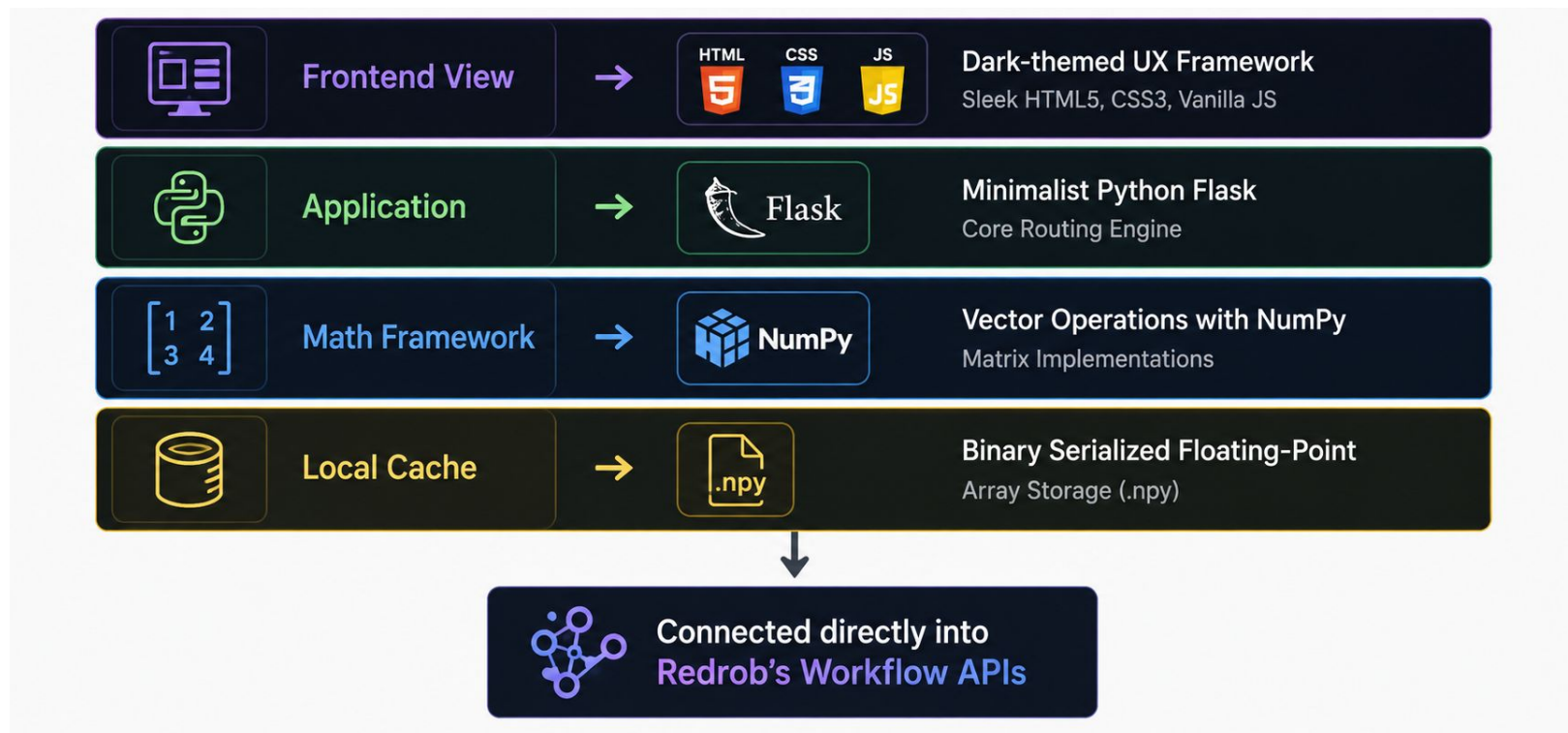
3. Cascading Algorithmic 7-Layer Filtering

- Title and Skill Gates: The array passes through chronological verification checks, applying steep math penalties if a candidate's title history reveals role mismatches or if claimed skills appear only in stuffed lists.
- Behavioral and Assessment Boosts: The pipeline adjusts scores using live database engagement records, applying multipliers for responsive user activity profiles and verified proctored evaluation scores.

4. Monotonic Score Normalization & Output

- The system feeds the finalized candidate composite score arrays into a dedicated monotonic mapping normalization algorithm.
- TrueBoard compresses the final values tightly between a stable 0.0 to 1.0 distribution index and outputs a perfectly sorted, verifiable list of elite talent directly to the recruiter's screen.

System Architecture



Results & Performance

What results or insights demonstrate ranking quality?

- **Elimination of Keyword Stuffing:** Unqualified profiles that copy-paste the JD into isolated skills lists are caught by the chronological work history scanner and penalized.
- **Seniority Guarding:** The Title Semantic Gate successfully identifies and filters out entry-level applicants applying for advanced roles, ensuring elite candidate matches.
- **Behavioral Trust Alignment:** High-throughput sorting matches raw text overlap with active user indicators, prioritizing highly responsive, assessment-verified talent.

How does your solution meet the challenge's runtime and compute constraints?

- **Sub-30 Second Processing:** Vector matrix multiplications compress the screening of 100,000+ candidates into a total execution time of under 30 seconds.
- **Localized Binary Caching:** Utilizing pre-calculated, lightweight .npy arrays eliminates heavy database parsing overhead and recurring cloud API latency.
- **Zero-Inference Token Overhead:** Running local open-source embeddings models removes dependencies on expensive third-party cloud computing resources.

Technologies Used

Python 3.9 & Flask

Why Selected: Selected to deliver a highly responsive, low-overhead microservice backend that handles input-output workflows with minimal execution latency.

HuggingFace Transformers (all-MiniLM-L6-v2)

Why Selected: Selected to run lightweight, open-source sentence embeddings locally, eliminating external cloud token reliance while maintaining deep contextual understanding.

NumPy Processing Core

Why Selected: Selected to execute ultra-fast, vectorized matrix multiplications and cosine similarity lookups over large candidate datasets in memory.

Binary .npy Serialized Caching

Why Selected: Selected to store pre-computed profile vector spaces locally, bypassing heavy database fetching overhead and enabling massive array loads instantly.

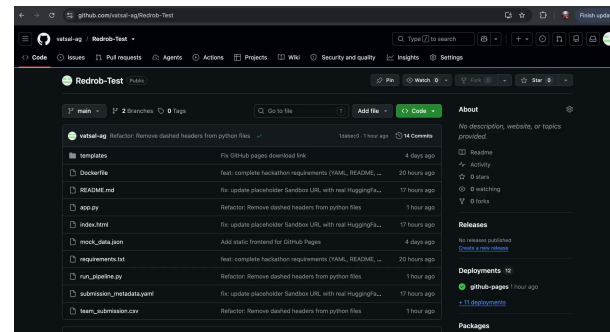
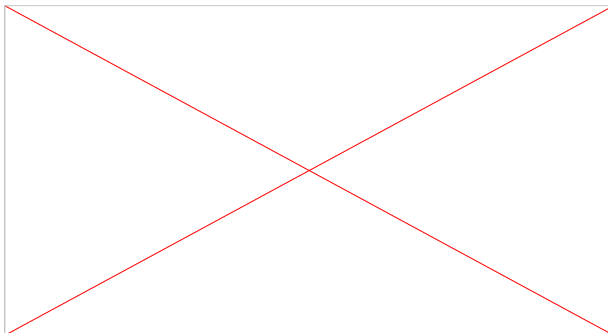
Vanilla HTML5 / CSS3 / JavaScript

Why Selected: Selected to build a sleek, distraction-free SaaS interface featuring an interactive dark mode aesthetic with zero dependencies on bulky frontend web frameworks.

Docker & HuggingFace Spaces

Why Selected: Selected to bundle the entire application pipeline into an isolated, containerized footprint for fast deployment and public, reproducible verification.

Submission Assets



- **templates:** Stores HTML templates to render web layouts.
- **Dockerfile:** Container blueprint to deploy seamlessly on HuggingFace Spaces.
- **README.md:** Core project documentation for system architecture and setup.
- **app.py:** Runs the Flask backend server to manage API routes.
- **index.html:** Powers the dark-themed recruiter dashboard and ranking tables.
- **mock_data.json:** Static candidate records for fast UI testing.
- **requirements.txt:** Lists python packages including Flask, Transformers, and NumPy.
- **run_pipeline.py:** Executes the core 7-layer vector math and ranking logic.
- **submission_metadata.yaml:** Holds hackathon tracking tags and track parameters.
- **team_submission.csv:** Spreadsheet containing finalized candidate evaluation outputs.



INDIA.RUNS

Build what next India runs on

THANK YOU